

CereVLA: Cerebellum-Inspired Consequence-Aware Residual Governance for Efficient Vision-Language-Action Execution

Shuai Zeng, Yuxuan Liang, Hangmiao Hu, Fobao Zhou, Zixiang Wang, Wenxi Hong, and Hang Zhao

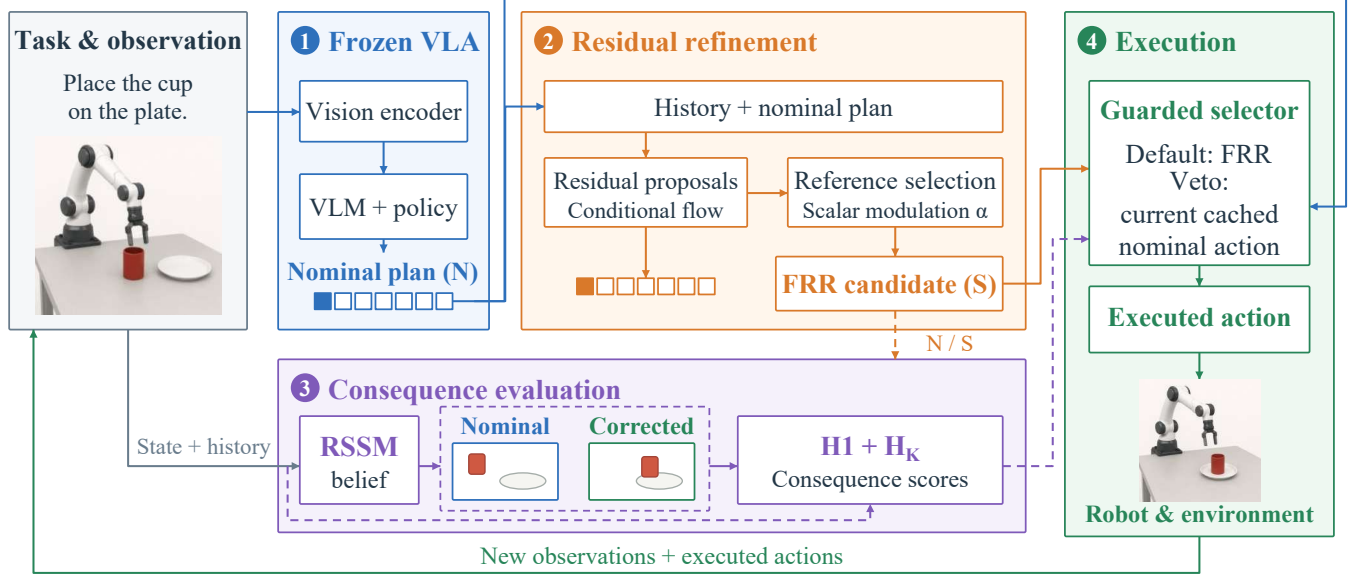


Fig. 1. Overview of CereVLA. FRR samples corrections, selects a candidate using an expert-supervised reference head, and applies learned scalar modulation. One-step evaluation (H_1) and history-aware classification (H_K) assess the correction. At each control tick, the governor retains the native FRR action by default and substitutes the current cached nominal action when veto conditions are met.

Abstract—Action-chunked vision-language-action (VLA) policies improve inference efficiency, but limited feedback within committed action chunks can lead to accumulated execution errors. Residual adaptation can correct such deviations without retraining the VLA; however, existing corrections are typically optimized for reference-action consistency without explicitly considering their downstream consequences. To address this limitation, we present Cerebellum-Inspired Consequence-Aware Residual Governance (CereVLA), a unified framework that integrates lightweight residual refinement and predictive consequence evaluation into frozen VLA execution. Corrective actions are first generated by flow-based residual refinement, and their short- and interval-horizon consequences are then evaluated by a recurrent state-space model and a history-aware classifier. Residual corrections predicted to be unfavorable are selectively suppressed by a lightweight governor. Comparisons with state-of-the-art methods on LIBERO-10 and LIBERO-GOAL demonstrate the effectiveness of CereVLA. On SO-101, CereVLA increases task success from 57.5% to 90.0% and reduces mean control steps by 19.6% among successful trials, relative to the frozen SmolVLA baseline.

I. INTRODUCTION

Vision-language-action (VLA) models are bringing robotic manipulation closer to general-purpose operation in house-

hold, industrial, laboratory, and service environments [1]–[6]. However, practical autonomy requires not only capable task reasoning, but also efficient and reliable physical execution. Repeatedly invoking a large VLA at every control step is computationally expensive, whereas executing long action chunks with limited feedback can accumulate errors as contact, tracking deviations, and environmental interactions alter the robot state.

Action chunking reduces inference cost by committing multiple actions from one policy query [7]–[9], but this also increases the mismatch between the observation used for planning and the state at execution time. Frequent replanning can restore feedback, yet introduces additional policy queries and does not consistently improve closed-loop behavior. Lightweight residual adaptation provides a more efficient alternative by locally correcting the nominal action without retraining the VLA. However, residual refinement is inherently double-edged: a correction may recover an execution error, but may also disturb a trajectory that would otherwise succeed.

Figure 2 illustrates the key limitation. A nominal plan may fail after execution disturbances; an imitation residual may reduce local deviation from an expert demonstration yet still lead to an unfavorable continuation, whereas an execution-effective correction may depart from the demon-

The authors are with the Hong Kong University of Science and Technology (Guangzhou), Guangzhou, China (e-mail: hangzhao@hkust-gz.edu.cn). Hang Zhao is the corresponding author.

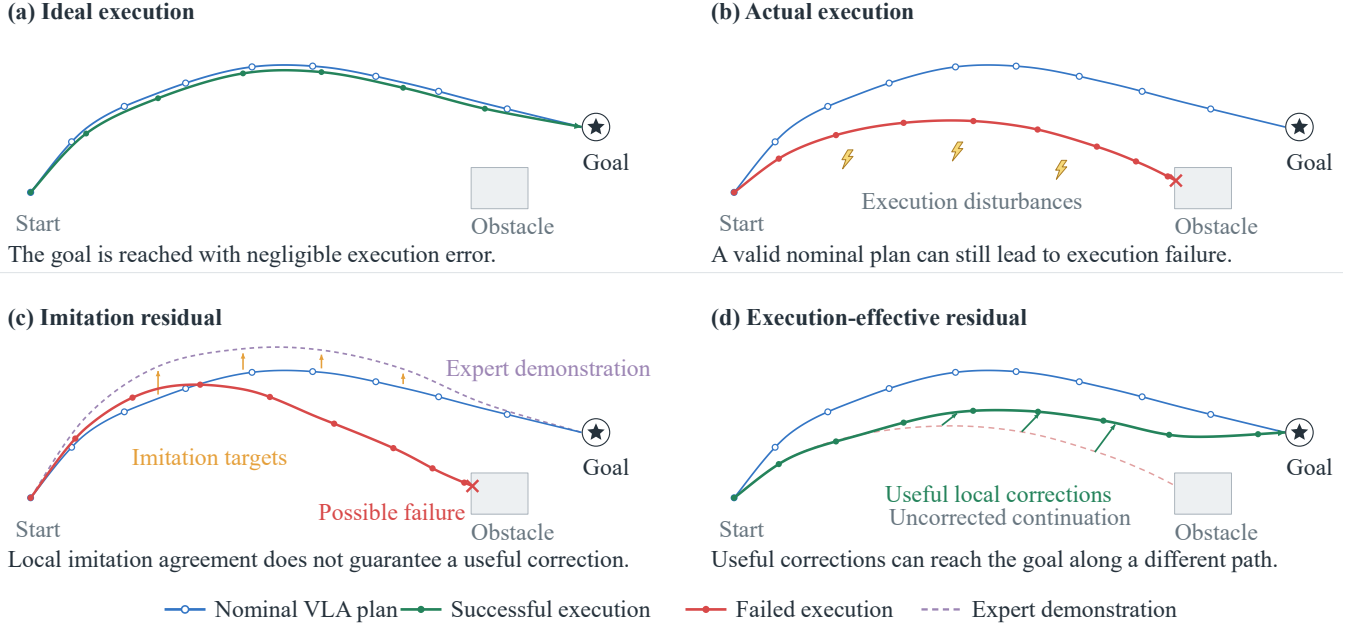


Fig. 2. Imitation residuals versus execution-effective corrections. (a) Ideal execution follows the nominal plan closely. (b) Disturbances can cause execution failure despite a valid nominal plan. (c) Better local agreement with an expert demonstration need not improve task completion. (d) Useful corrections may reach the goal along a different trajectory. Blue denotes nominal plans, green successful executions, red failed executions, and dashed purple an expert demonstration.

strated trajectory while improving task progress. Thus, the central problem is not simply how to generate a residual, but whether a generated correction should be executed. In other words, *imitation agreement does not necessarily imply execution utility*. Residual generation and reactive replanning address how to modify an action or when to query the policy. CereVLA addresses the complementary decision of whether the residual-corrected action should be executed instead of the nominal action.

Inspired by the cerebellum’s predictive role in estimating the consequences of motor commands [10], [11], we propose **CereVLA**, a consequence-aware execution architecture for frozen VLA policies. As shown in Fig. 1, CereVLA separates fast residual generation from predictive residual qualification. A flow-based residual refinement (FRR) module produces lightweight online corrections from recent execution history and the cached VLA plan. CereVLA then compares nominal and corrected alternatives at complementary temporal horizons using an action-conditioned recurrent state-space model and a history-aware consequence classifier. A conservative governor suppresses a residual only when the predicted evidence indicates an adverse consequence, while otherwise preserving native residual execution and the original VLA replanning schedule. The nominal continuation serves only as a relative reference and is not assumed to be globally optimal or safe.

The main contributions of this work are summarized as follows:

- We propose **CereVLA**, a unified cerebellum-inspired execution framework that integrates flow-based residual refinement, predictive consequence evaluation, and restricted-authority execution governance around frozen

VLA policies.

- Within CereVLA, one-step consequence evaluation (H_1) and history-aware interval classification (H_K) provide complementary evidence for retaining or suppressing native FRR corrections.
- We evaluate CereVLA on LIBERO with two frozen VLA backbones and on a physical SO-101, reporting recovery–harm trade-offs in simulation and task performance and motion quality on hardware.

II. RELATED WORK

A. Action-Chunked VLA Execution

Action chunking reduces repeated policy inference by predicting temporally extended action sequences, as adopted by ACT, Diffusion Policy, and OpenVLA-OFT [7]–[9]. Recent methods further improve execution-time feedback. Real-Time Chunking overlaps inference with execution [12], while VLA-Corrector detects persistent visual deviations and adaptively truncates or replans action chunks [13]. These methods primarily modify when the policy is queried or replanned. CereVLA instead retains the VLA replanning schedule and addresses a complementary problem: whether an already available residual correction should be allowed to modify the committed trajectory.

B. Residual Adaptation

Residual policies augment an existing controller through additive corrections, avoiding full policy replacement or retraining [14], [15]. In supervised settings, residuals are naturally learned from expert–nominal action differences. Such objectives provide effective correction targets, but do not explicitly encode whether a particular residual improves

downstream closed-loop execution. CereVLA therefore decouples residual proposal from residual qualification: FRR generates candidate corrections, while an independent consequence layer determines whether the native residual should be retained or suppressed.

C. Predictive Consequence Modeling

Predictive models have long been used to compensate for delayed or incomplete feedback in model-based and teleoperation control [16], [17]. Learned world models extend this idea to high-dimensional robotic decision making; Dreamer learns latent dynamics for behavior optimization [18], while Generative Predictive Control evaluates or refines proposals from frozen generative policies [19]. VIP and LIV provide complementary visual representations of task progress [20], [21]. Unlike methods that perform full trajectory imagination or policy replanning, CereVLA uses predictive modeling only to compare the relative consequences of nominal and residual actions. This narrower role enables lightweight execution-time governance without modifying the frozen VLA.

III. METHODOLOGY

CereVLA evaluates residual corrections before they modify the execution of a frozen vision-language-action (VLA) policy. The framework combines flow-based residual refinement (FRR), one-step consequence evaluation (H_1), interval-level consequence classification (H_K), and an execution governor. We distinguish nominal VLA execution N , native FRR execution S , and governed execution G . The VLA replanning interval is denoted by R , whereas K denotes the offline consequence-supervision horizon of H_K . The reported implementation uses $R = K = 5$, and the corresponding interval classifier is denoted by H_5 . Importantly, the online governor is evaluated at every control tick; K does not specify the duration of an online fallback.

A. Frozen Planning and Residual Refinement

The VLA generates a nominal action chunk every R control steps. Between replanning events, FRR uses the latest feedback to refine the current action. Its input is

$$c_t^{\text{FRR}} = (s_{t-7:t}, \bar{\mathbf{A}}_t^N, a_t^N, e_{\text{task}}, \rho_t, z_t^{\text{vis}}), \quad (1)$$

where $s_t \in \mathbb{R}^8$ contains end-effector (EEF) position, an axis-angle orientation vector, and two gripper coordinates. The nominal plan $\bar{\mathbf{A}}_t^N \in \mathbb{R}^{50 \times 7}$ is shifted to the current step and zero-padded, with first action a_t^N . The task code e_{task} is a one-hot task identifier, and ρ_t is plan age divided by $R-1$. Visual context $z_t^{\text{vis}} \in \mathbb{R}^{1920}$ concatenates mean-pooled frozen-image-encoder features from two camera views.

FRR learns aligned expert–nominal residuals through conditional flow matching [22]:

$$\begin{aligned} \mathbf{R}^* &= \text{clip}((\mathbf{A}^E - \bar{\mathbf{A}}^N) \odot \mathbf{s}_\Delta, -1, 1), \\ \mathbf{R}_u &= (1 - u)\epsilon + u\mathbf{R}^*, \\ \mathcal{L}_{\text{flow}} &= \mathbb{E}[\text{MSE}_M(v_\theta(\mathbf{R}_u, u, c_t^{\text{FRR}}, m_t), \mathbf{R}^* - \epsilon)]. \end{aligned} \quad (2)$$

Here $\mathbf{A}^E$ denotes the aligned expert action sequence, $\epsilon \sim \mathcal{N}(0, I)$, $u \sim \mathcal{U}(0, 1)$, and $\mathbf{s}_\Delta$ contains the stored per-action residual scales. The mask M combines chunk validity and correction supervision. The binary generation mode m_t uses correction labels during training and is determined by auxiliary risk/recovery heads during inference. Auxiliary objectives supervise the expert-reference and local-dynamics heads, together with risk, progress, recovery, replanning, and value estimates; flow-velocity regularization is also applied.

Four residual candidates are sampled using four Heun integration steps. Each normalized sample $\hat{\mathbf{R}}_t^{(i)}$ is converted to an action-space residual,

$$\Delta \mathbf{A}_t^{(i)} = \text{clip}(\hat{\mathbf{R}}_t^{(i)}, -1, 1) \odot \mathbf{s}_\Delta. \quad (3)$$

Each corrected plan $\bar{\mathbf{A}}_t^N + \Delta \mathbf{A}_t^{(i)}$ is clipped to the action bounds before reference-based selection. The candidate set also includes the unmodified nominal plan with zero residual. An expert-supervised reference head selects the candidate with the closest first action:

$$\begin{aligned} a_t^{\text{ref}} &= \tanh f_{\text{ref}}(c_t^{\text{FRR}}), \\ i^* &= \arg \min_i \|a_{t,0}^{(i)} - a_t^{\text{ref}}\|_2^2. \end{aligned} \quad (4)$$

Let $\Delta a_{t,0}^{(i)}$ denote the first action-space residual of candidate i . The native command is

$$a_t^S = a_t^N + \alpha_t \Delta a_{t,0}^{(i^*)}, \quad \alpha_t = \text{sigmoid}(f_\alpha(c_t^{\text{FRR}})). \quad (5)$$

Only the current command is used for execution; residual proposals are recomputed at the next control step from updated feedback. If the native output is non-finite, the controller executes a_t^N instead. CereVLA preserves this native FRR computation and evaluates the effective correction $\Delta a_t = a_t^S - a_t^N$ before execution.

B. Task Relations and One-Step Consequences

For task k , the relation map $r_t = \Phi_k(X_t)$ encodes object–EEF and object–goal offsets, together with articulated-joint error when applicable. Here X_t denotes the underlying robot–environment state. The current task stage determines which relation is relevant to progress.

A recurrent state-space model (RSSM) [23], [24] estimates action-dependent changes in these relations. Its input x_t includes robot state, the previous executed action, the current nominal action, a five-step nominal-plan window, plan phase and age, task code, and visual context. With $e_t = E_\psi(x_t)$, online filtering uses the posterior mean:

$$\bar{z}_t = \mu_{q,\psi}(h_t, e_t), \quad h_{t+1} = \text{GRU}_\psi([e_t, \bar{z}_t, a_t^{\text{exec}}], h_t). \quad (6)$$

The RSSM is trained with stochastic latents using reconstruction, robot- and relation-change prediction, latent KL, and auxiliary predicate objectives. Predicted relation changes provide the consequences used for action comparison.

At every control tick with a nontrivial native correction, nominal and native FRR actions are evaluated from the same filtered belief and observation encoding. Four shared prior-noise samples produce paired predictions, which are

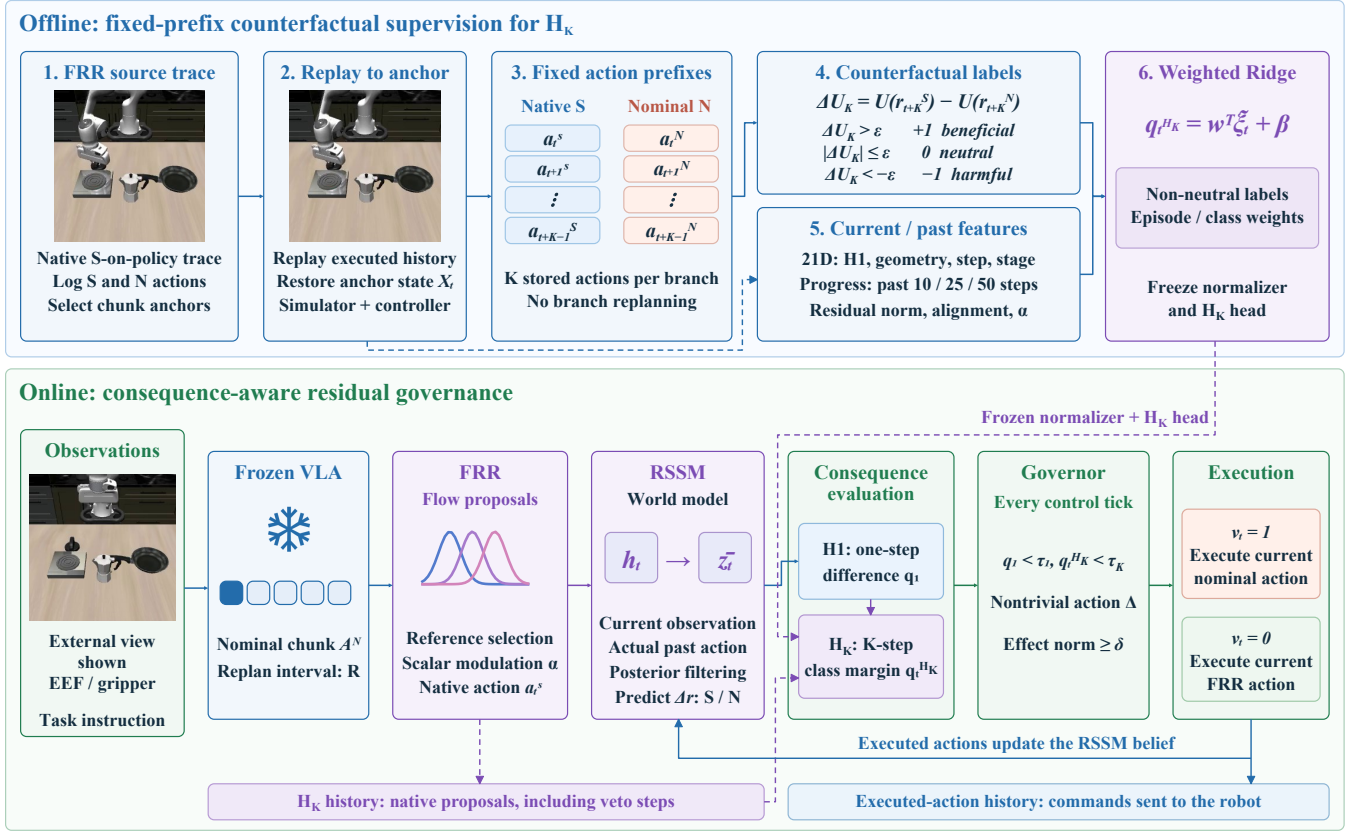


Fig. 3. Offline supervision and online execution in CereVLA. Recorded native and nominal K -action prefixes are replayed from the same anchor without branch replanning. Endpoint utility differences provide signed labels for H_K . Online inference uses current and past features with the frozen classifier and its fitted normalizer; recorded future actions are used only for offline labeling. The governor recomputes its decision at every control tick. Each veto substitutes the current cached nominal action only and may recur without an episode-wide count limit. R is the VLA replanning interval; H_5 is the evaluated $K = 5$ instance.

transformed back to relation units and averaged. Holding the current task stage and target fixed, H_1 computes

$$\begin{aligned} \hat{r}_{t+1}^B &= r_t + \Delta \hat{r}_t^B, \quad B \in \{N, S\}, \\ q_t^{H_1} &= U_t(\hat{r}_{t+1}^S) - U_t(\hat{r}_{t+1}^N). \end{aligned} \quad (7)$$

The utility U_t is the negative object–EEF distance during approach, negative object–goal distance during placement, and negative absolute joint error during closure. Thus, $q_t^{H_1} < 0$ indicates a predicted adverse one-step consequence of the native correction.

The paired formulation emphasizes *relative* consequence prediction. Let e_t^S and e_t^N denote the utility-prediction errors of the native and nominal branches, and let $\Delta U_{1,t}$ be their true one-step utility difference. Then

$$q_t^{H_1} - \Delta U_{1,t} = e_t^S - e_t^N. \quad (8)$$

Hence the comparison error depends on the difference between branch-wise prediction errors rather than on either absolute prediction error alone. Sharing the anchor, belief, predictor, and stochastic samples encourages common-mode errors to cancel, although such cancellation is not guaranteed.

C. Interval-Level Counterfactual Supervision

H_K complements the one-step comparison by classifying consequences over a fixed K -action horizon. Source trajec-

tories are collected under frozen native FRR execution. For each selected anchor, two branches replay the same recorded execution history and then apply either the recorded native or nominal prefix:

$$\begin{aligned} r_{t+K}^B &= \Phi_k(\mathcal{T}_K(X_t, \mathbf{A}_{t,\text{src}}^B)), \quad B \in \{N, S\}, \\ \Delta U_{K,t} &= U_t(r_{t+K}^S) - U_t(r_{t+K}^N). \end{aligned} \quad (9)$$

Here $\mathcal{T}_K$ denotes evolution under K prescribed actions, with $\mathbf{A}_{t,\text{src}}^B$ containing the source actions from t through $t + K - 1$. Both branches keep the recorded prefixes fixed, so the labels characterize these prefixes rather than feedback policies recomputed along each branch. The target is $y_t = +1$ when $\Delta U_{K,t} > \epsilon$, $y_t = -1$ when $\Delta U_{K,t} < -\epsilon$, and $y_t = 0$ otherwise. Neutral records are retained but excluded from classifier fitting and classwise decision metrics.

In the implemented five-step instance, the feature vector $\xi_t \in \mathbb{R}^{21}$ contains the H_1 score, the log-magnitude of the predicted relation difference, current task distances and closure error, normalized episode step, stage progress over 10/25/50-step windows, window means of residual norm, nominal–native cosine alignment and α_t , and the task-stage one-hot code. Motion-history features use native FRR proposals, including during vetoed intervals; the RSSM instead updates its belief using the actions actually executed.

For standardized features $\tilde{\xi}_i$, H_K is fitted using a weighted

ridge classification surrogate:

$$(w^*, \beta^*) = \arg \min_{w, \beta} \sum_{i: y_i \neq 0} \omega_i (y_i - w^\top \tilde{\xi}_i - \beta)^2 + \lambda \|w\|_2^2 + 10^{-8} \beta^2, \\ q_t^{HK} = (w^*)^\top \tilde{\xi}_t + \beta^*. \quad (10)$$

Sample weights combine inverse per-episode counts with class-frequency weighting and are normalized to unit mean. Lower scores indicate stronger evidence of a harmful prefix. Unlike H_1 , H_K outputs a signed classification margin rather than a predicted utility difference. At deployment, it evaluates current and past features without an online K -step world-model rollout.

Counterfactual decision interpretation. The offline supervision in Eq. (9) can be interpreted as a selection problem between native and nominal fixed prefixes. For an idealized selector, let $v \in \{0, 1\}$ denote selecting the nominal prefix ($v = 1$) rather than the native FRR prefix ($v = 0$). Relative to an oracle that always selects the higher-utility branch, the local selection regret is

$$\mathcal{R}_t(v) = v[\Delta U_{K,t}]_+ + (1 - v)[- \Delta U_{K,t}]_+, \\ [x]_+ \triangleq \max(x, 0). \quad (11)$$

Thus, an incorrect rejection of a strongly beneficial residual can incur a larger consequence loss than failing to reject a mildly harmful residual. This perspective motivates consequence-aware qualification based on both sign and consequence magnitude rather than classification accuracy alone.

Equation (11) characterizes only the fixed-prefix counterfactual supervision used to train H_K . It is not an equality for the deployed CereVLA trajectory: online execution re-evaluates the decision at every control tick and does not commit to either fixed K -step branch.

D. Execution Governance

At every control tick, CereVLA combines the short-horizon H_1 evidence and the interval-level H_K margin to determine whether the current native FRR correction should be retained. Let $d_t \in \{0, 1\}$ indicate that the native and nominal actions differ by a nontrivial amount, and define the predicted relative relation change

$$\Delta \hat{r}_t^{\text{CF}} = \Delta \hat{r}_t^S - \Delta \hat{r}_t^N. \quad (12)$$

The veto rule is

$$v_t = d_t \mathbf{1}[q_t^{H_1} < \tau_1] \mathbf{1}[q_t^{HK} < \tau_K] \mathbf{1}[\|\Delta \hat{r}_t^{\text{CF}}\|_2 \geq \delta]. \quad (13)$$

When $v_t = 0$, CereVLA retains the native FRR command. When $v_t = 1$, only the current residual correction is suppressed and the corresponding nominal action is executed:

$$a_t^G = (1 - v_t) a_t^S + v_t a_t^N. \quad (14)$$

The decision is recomputed at the next control tick using the updated observation, FRR proposal, and executed-action history. Consequently, vetoes are repeatable and there is no episode-level intervention budget. Each veto acts only on the

current action and does not lock execution to a nominal K -step prefix. The horizon K therefore defines the interval used to construct supervision for H_K , whereas online governance remains temporally local and feedback-driven.

This design gives the consequence layer restricted authority: it may suppress the current residual and revert to the nominal action, but it cannot generate an arbitrary alternative command or alter the VLA replanning schedule. The nominal action is therefore used as a relative execution reference rather than as a certified safe controller.

IV. EXPERIMENTS

A. Experimental Setup

We evaluate frozen $\pi_{0.5}$ and SmolVLA policies on LIBERO-10 and LIBERO-GOAL [25]. Episodes terminate at success or the task horizon; failures and timeouts remain in the success-rate denominator. The standard policy replans every $R = 5$ control steps, while FRR updates the current correction at each step. The evaluated interval classifier is H_5 , with $K = 5$. Paired comparisons use matched task initial conditions. The current workstation contains an NVIDIA RTX 4090 GPU and an Intel Core i7-13700 CPU.

We compare the nominal *Base* policy, *Frequent replanning* at every control step, native *FRR*, its *modulation variant*, and *CereVLA*. VLA-Corrector [13] and FutureRTC [26] provide additional execution-correction comparisons under their evaluated configurations. FRR tests residual adaptation, whereas CereVLA additionally governs whether the proposed correction is used.

We report suite success rates and their equal-weight average. Relative to the corresponding Base execution, Recovery (R) and Harm (H) are the percentages of all paired episodes that change from failure to success and success to failure, respectively. If n_{ij} counts reference/method outcomes (i, j) and $n = \sum_{i,j} n_{ij}$, then $R = 100n_{01}/n$, $H = 100n_{10}/n$, and $\text{Net} = R - H$. Net is the paired success difference in percentage points.

B. Overall Performance

Table I shows that CereVLA has the highest reported equal-suite mean for each backbone, reaching 95.35% with $\pi_{0.5}$ and 64.00% with SmolVLA. Both improve over Base and native FRR on the two suites. The advantage is more pronounced with SmolVLA; for $\pi_{0.5}$, the mean gain over FRR is modest, and FutureRTC retains the highest LIBERO-GOAL success rate. The results therefore favor CereVLA in aggregate without implying superiority on every backbone-suite combination.

The paired Recovery–Harm analysis provides a more direct view of execution reliability than aggregate success alone. Native FRR exhibits a clear recovery–harm trade-off: residual corrections can recover failed nominal executions, but the same mechanism can also convert previously successful executions into failures. CereVLA shifts this balance toward more selective intervention. Across all four backbone-suite combinations, CereVLA achieves a positive Net gain that exceeds native FRR, indicating that its improvement

TABLE I

TASK SUCCESS AND PAIRED RECOVERY-HARM ANALYSIS ON LIBERO. SUCCESS, R , AND H ARE PERCENTAGES; NET IS IN PERCENTAGE POINTS. AVG. IS THE EQUAL-SUITE MEAN; PAIRED OUTCOMES USE BASE AS REFERENCE.

Backbone	Method	LIBERO-10	LIBERO-GOAL	Avg.	LI0 R/H/Net	GOAL R/H/Net
$\pi_{0.5}$	Base	92.00	96.00	94.00	—	—
	Frequent replanning	69.50	89.50	79.50	3.50/26.00/ − 22.50	2.00/8.50/ − 6.50
	FRR	93.00	97.00	95.00	6.50/5.50/ + 1.00	2.00/1.00/ + 1.00
	FRR (modulation variant)	93.00	97.50	95.25	4.50/3.50/ + 1.00	2.50/1.00/ + 1.50
	VLA-Corrector	92.00	96.50	94.25	3.50/3.50/0.00	2.50/2.00/ + 0.50
	FutureRTC	91.50	98.50	95.00	2.00/2.50/ − 0.50	3.50/1.00/ + 2.50
	CereVLA	93.20	97.50	95.35	4.40/3.20/ + 1.20	3.25/1.75/ + 1.50
SmolVLA	Base	46.25	68.00	57.13	—	—
	Frequent replanning	27.50	79.50	53.50	3.25/22.00/ − 18.75	22.00/10.50/ + 11.50
	FRR	46.50	72.00	59.25	13.25/13.00/ + 0.25	11.00/7.00/ + 4.00
	FRR (modulation variant)	47.00	72.00	59.50	6.25/5.50/ + 0.75	11.00/7.00/ + 4.00
	VLA-Corrector	20.50	50.50	35.50	10.50/36.25/ − 25.75	22.50/40.00/ − 17.50
	FutureRTC	20.00	49.50	34.75	13.00/39.25/ − 26.25	15.00/33.50/ − 18.50
	CereVLA	55.00	73.00	64.00	15.50/6.75/ + 8.75	9.50/4.50/ + 5.00

Bold marks the highest success rate per column within each backbone.

is not driven simply by applying more corrections, but by producing a more favorable balance between recovered failures and newly introduced failures.

This effect is particularly evident with SmolVLA. On LIBERO-10, CereVLA substantially reduces Harm relative to native FRR while simultaneously increasing Recovery, yielding a markedly larger positive Net gain. On LIBERO-GOAL, CereVLA again reduces Harm while maintaining sufficient Recovery to improve the paired Net outcome. For $\pi_{0.5}$, the gain is more moderate and does not arise from uniformly minimizing Harm. Instead, CereVLA maintains a favorable recovery-harm balance, producing a higher positive Net outcome on both suites. This distinction is important: the governor is designed to improve closed-loop utility rather than to suppress residual activity as aggressively as possible. These results support the intended role of consequence-aware governance: beneficial residual behavior is largely retained, whereas corrections with adverse execution consequences are more selectively suppressed.

C. Component Analysis and Motion Quality

We use LIBERO to examine local consequence discrimination and execution quality with SmolVLA + FRR. Beneficial retention (BR) is the fraction of beneficial labeled anchors retained; harmful rejection (HR) is the fraction of harmful anchors vetoed. Veto precision is the harmful fraction among vetoed non-neutral anchors. These local metrics precede execution governance and differ from episode-level Recovery and Harm.

Table II shows that adding H5 shifts the evaluator toward more conservative intervention: beneficial retention and veto precision improve, while harmful rejection decreases. This trade-off is intentional, since CereVLA prioritizes preserving useful residuals rather than maximizing veto frequency.

More importantly, H1+H5 and the Full Governor share the same local BR, HR, and veto-precision scores, yet produce markedly different closed-loop behavior. The Full Governor is the only configuration that simultaneously reduces completion steps, action variation, and both translational and

rotational EEF variation. This indicates that consequence prediction alone is insufficient; bounded governance is required to translate local evidence into efficient and smooth execution.

Motion metrics use third finite differences of the logged executed actions and EEF trajectories. Action D3 RMS is computed over all seven command dimensions without time scaling. EEF position and axis-angle rotation-vector differences are divided by Δt^3 with $\Delta t = 0.05$ s, where the orientation measure is treated as a coordinate-based smoothness proxy. Metrics are averaged over the common-success subset, and relative changes are computed from these episode-level means.

a) Computational overhead: The CereVLA execution layer introduces 8.06M additional parameters. This corresponds to 1.33% of the 604.93M-parameter frozen SmolVLA backbone and approximately 0.27% of a 3B-parameter $\pi_{0.5}$ backbone. On the RTX 4090 workstation, FRR inference requires 8.2 ms, while the consequence module adds 2.74 ms per tick. These timings exclude VLA inference and therefore should not be interpreted as total control-loop latency.

D. Real-Robot Evaluation

Setup and metrics. We evaluate CereVLA on a physical SO-101 using the instruction “Grab the orange to basket.” Before each trial, the orange is randomly placed within a predefined reachable tabletop workspace, while the target basket remains fixed. The robot uses a front-view camera, a wrist camera, and joint-state feedback. All methods share the same frozen SmolVLA checkpoint.

We compare *Base*, native *FRR*, and the complete *CereVLA*. FRR recomputes the current residual from updated feedback at every control cycle, while SmolVLA retains its native 50-action queue. CereVLA further evaluates nominal and residual-corrected alternatives using short- and interval-horizon consequence evidence and permits repeated vetoes without an episode-level budget. Each method is evaluated over 40 trials. The target control rate is 30 Hz, with a 20 s / 700-step task limit. Success requires placing the orange

TABLE II

EVALUATOR COMPONENT ANALYSIS ON LIBERO WITH SMOLVLA. LOCAL BR/HR METRICS EVALUATE CONSEQUENCE-BASED DECISIONS, WHILE MOTION METRICS AND EVALUATOR OVERHEAD ARE MEASURED RELATIVE TO NATIVE FRR EXECUTION.

Evaluator	BR (%)	HR (%)	Veto Prec.(%)	Steps Δ (%)	Action D3 Δ (%)	EEF Pos. Jerk Δ (%)	Rot. Vec. D3 Δ (%)	Eval. Overhead(ms)
H1 geometric	100.0	100.0	100.0	+38.67	-18.30	-4.10	-16.26	0.002
H1 learned	75.0	80.0	80.0	+18.29	-30.45	-6.76	-10.40	2.006
H1 + H5	100.0	53.3	100.0	+9.37	-16.61	+2.30	+7.22	2.733
+ Continuation	100.0	53.3	100.0	-2.34	+18.21	+19.72	+19.57	2.737
Full (+ Governor)	100.0	53.3	100.0	-9.35	-3.68	-1.71	-4.92	2.740

TABLE III

REAL-ROBOT PERFORMANCE ON THE SO-101 ORANGE PICK-AND-PLACE TASK. EACH METHOD USES 40 TRIALS. STEPS AND MOTION METRICS ARE REPORTED OVER SUCCESSFUL TRIALS AS MEAN $\pm$ SAMPLE STANDARD DEVIATION.

Method	Success	Steps	Span (s)	Command Jerk Proxy	Acceleration RMS (rad/s ²)
SmolVLA	57.5%	354.70 $\pm$ 89.78	12.71 $\pm$ 3.23	3.462 $\pm$ 0.446	4.383 $\pm$ 0.465
SmolVLA + FRR	82.5%	322.21 $\pm$ 65.55	11.66 $\pm$ 2.30	4.319 $\pm$ 0.406	4.588 $\pm$ 0.496
CereVLA	90.0%	285.34 $\pm$ 30.45	9.97 $\pm$ 1.06	4.020 $\pm$ 0.333	4.329 $\pm$ 0.498

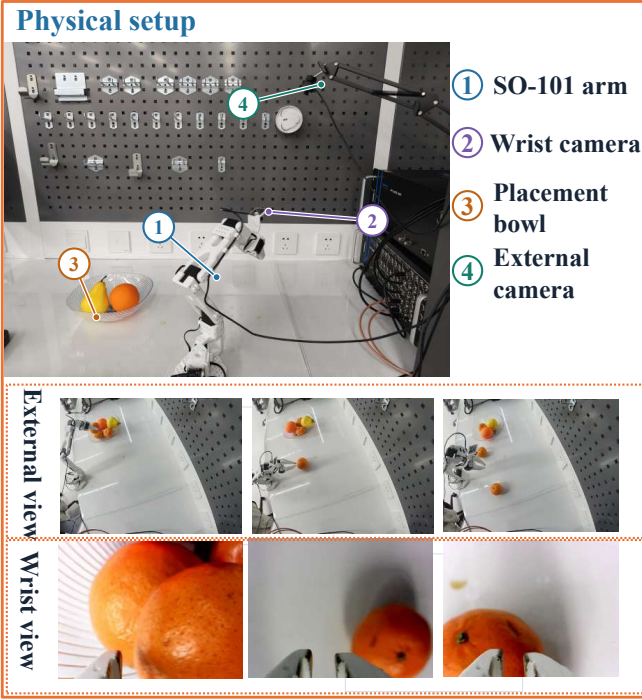


Fig. 4. Physical SO-101 setup for orange pick-and-place with randomized object placement, wrist camera, and external camera.

inside the basket. The three groups are collected in separate batches and are therefore not strictly paired.

Task efficiency is measured by successful-trial steps and recorded execution span. Motion quality is evaluated using the RMS third finite difference of the five arm-joint commands (command jerk proxy) and the time-weighted RMS of measured joint-acceleration magnitude. Joint velocity and acceleration are computed offline from measured joint positions and physical timestamps.

Results. Native FRR substantially improves real-robot success over the frozen SmolVLA baseline, but also increases command variation. CereVLA further raises success to

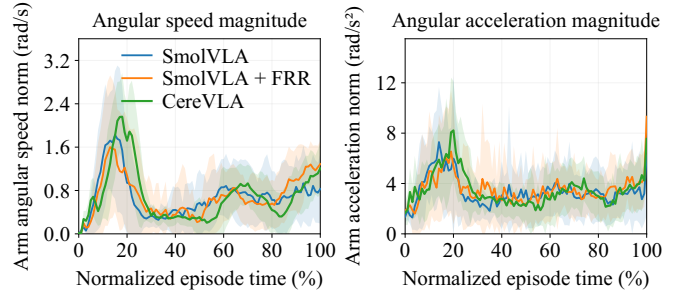


Fig. 5. Measured whole-arm angular-speed and angular-acceleration magnitudes over normalized successful-episode time. Curves show the across-episode mean and shaded regions indicate one standard deviation.

90.0%, while reducing mean completion steps and execution span by 11.4% and 14.5%, respectively, relative to FRR. It also reduces the FRR-induced command jerk by 6.9% and measured acceleration RMS by 5.6%. Thus, CereVLA preserves the recovery capability of residual refinement while improving execution efficiency and mitigating part of its additional motion variation.

Figure 5 provides the corresponding time-resolved motion comparison. The nominal policy retains the lowest command jerk, whereas CereVLA reduces the additional variation introduced by FRR while achieving higher task success. Successful CereVLA runs operate at an effective observation rate of approximately 28.5 Hz under the configured 30 Hz target. Since the three groups are not paired by initial state, these results are interpreted as deployment-level evidence rather than a strictly paired causal comparison.

V. CONCLUSION

This work presents **CereVLA**, a consequence-aware execution framework that combines online residual refinement with predictive qualification for frozen action-chunked VLA policies. Across LIBERO-10 and LIBERO-GOAL, CereVLA improves task success on two VLA backbones and achieves a more favorable Recovery–Harm balance than native residual

execution. Real-robot experiments on a low-cost SO-101 further increase success from 82.5% with FRR to 90.0%, while reducing completion steps and part of the residual-induced motion variation. These results show that reliable VLA execution depends not only on generating corrective residuals, but also on predicting when those corrections should be applied. Future work will explore longer-horizon world-model reasoning and reinforcement-learning signals for optimizing residual generation toward closed-loop task utility.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [2] B. Zitkovich, T. Yu, S. Xu *et al.*, “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of the 7th Conference on Robot Learning (CoRL)*, vol. 229. PMLR, 2023, pp. 2165–2183.
- [3] M. J. Kim, K. Pertsch, S. Karamcheti *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Proceedings of the 8th Conference on Robot Learning (CoRL)*, vol. 270. PMLR, 2025, pp. 2679–2713.
- [4] D. Ghosh, H. R. Walke, K. Pertsch *et al.*, “Octo: An open-source generalist robot policy,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2024.
- [5] K. Black, N. Brown, D. Driess *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [6] K. Black, N. Brown, J. Darpinian *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of the 9th Conference on Robot Learning (CoRL)*, vol. 305. PMLR, 2025, pp. 17–40.
- [7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2023.
- [8] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.
- [9] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Proceedings of Robotics: Science and Systems (RSS)*, 2025.
- [10] D. M. Wolpert, R. C. Miall, and M. Kawato, “Internal models in the cerebellum,” *Trends in Cognitive Sciences*, vol. 2, no. 9, pp. 338–347, 1998.
- [11] I. Abadía, F. Naveros, E. Ros, R. R. Carrillo, and N. R. Luque, “A cerebellar-based solution to the nondeterministic time delay problem in robotic control,” *Science Robotics*, vol. 6, no. 58, p. eabf2756, 2021.
- [12] K. Black, M. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38. Curran Associates, Inc., 2025, pp. 33 383–33 407.
- [13] Y. Pan, M. Pan, Q. Lu, J. Huang, M. Zhang, S. Huang, X. Li, J. Zhang, Y. Shen, X. Zhang, and W. Zhang, “VLA-Corrector: Lightweight detect-and-correct inference for adaptive action horizon,” *arXiv preprint arXiv:2607.01804*, 2026.
- [14] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, “Residual policy learning,” *arXiv preprint arXiv:1812.06298*, 2018.
- [15] T. Johannink, S. Bahl, A. Nair, J. Luo, A. Kumar, M. Loskyll, J. A. Ojeda, E. Solowjow, and S. Levine, “Residual reinforcement learning for robot control,” in *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 2019, pp. 6023–6029.
- [16] A. C. Smith and K. Hashtrudi-Zaad, “Smith predictor type control architectures for time delayed teleoperation,” *The International Journal of Robotics Research*, vol. 25, no. 8, pp. 797–818, 2006.
- [17] S. Sirouspour and A. Shahdi, “Model predictive control for transparent teleoperation under communication time delay,” *IEEE Transactions on Robotics*, vol. 22, no. 6, pp. 1131–1145, 2006.
- [18] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [19] H. Qi, H. Yin, A. Zhu, Y. Du, and H. Yang, “Inference-time enhancement of generative robot policies via predictive world modeling,” *IEEE Robotics and Automation Letters*, vol. 11, no. 5, pp. 5534–5541, 2026.
- [20] Y. J. Ma, S. Sodhani, D. Jayaraman, O. Bastani, V. Kumar, and A. Zhang, “VIP: Towards universal visual reward and representation via value-implicit pre-training,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [21] Y. J. Ma, V. Kumar, A. Zhang, O. Bastani, and D. Jayaraman, “LIV: Language-image representations and rewards for robotic control,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, vol. 202. PMLR, 2023, pp. 23 301–23 320.
- [22] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [23] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, “Learning latent dynamics for planning from pixels,” in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, vol. 97. PMLR, 2019, pp. 2555–2565.
- [24] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, “Mastering diverse control tasks through world models,” *Nature*, vol. 640, no. 8059, pp. 647–653, 2025.
- [25] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36. Curran Associates, Inc., 2023, pp. 44 776–44 791.
- [26] H. Jiang, Y. Zou, B. Liang, B. Liu, F. Meng, and S. Liu, “Futurertc: Real-time robot execution with anticipatory-conditioned action chunking,” *arXiv preprint arXiv:2607.24008*, 2026.